

Betrayal Game — Wave 3: UX & Playtest Readiness

Replit Agent Engineering Prompts

These prompts make the game playtest-ready for a real group. Run them in order — they build on each other. No new mechanics are introduced. This wave is entirely about clarity, confidence, and feel.

Prerequisites: Wave 1 complete (WS Router + SQLite extracted). Wave 2 is not required before starting Wave 3.

PROMPT 1: Phase Clarity & Persistent HUD

You are working on **Betrayal Game** (Node.js + TypeScript backend, React 19 + Vite frontend, CSS Modules). The game has a fully functional core loop across these phases:

LOBBY , ROLE_ASSIGN , ROLE_REVEAL , ROUNDTABLE , VOTING , REVOTE , VOTE_REVEAL ,
TIE_DETECTED , TIEBREAKER_REVEAL , BANISH_REVEAL , NIGHT , MORNING , CHALLENGE ,
CHALLENGE_RESULT , GAME_END .

The problem

Players currently have no persistent visual anchor. Each phase renders a different component with its own layout, and there is no consistent element that tells a player: who they are, what their role is, who is alive, and what is happening right now. First-time players in a live game feel lost between phases.

What needs to be done

1. Persistent HUD component

Create `client/src/components/HUD.tsx` and `HUD.module.css`.

The HUD is a fixed bar that renders on top of every in-game screen (all phases except `LOBBY` , `ROLE_ASSIGN` , and `GAME_END`). It has two zones:

Top bar (always visible, ~56px tall):

- Left: The player's own name + role badge. Role badge: a small pill showing "TRAITOR"

(crimson #8b1a1a) or "FAITHFUL" (slate #2a4a6b). Dead players show their name struck-through and greyed out.

- Centre: Current phase label — human-readable, not the raw enum. Map as follows:
 - `ROLE_REVEAL` → "Your Role"
 - `ROUNDTABLE` → "Round [N] — Discussion"
 - `VOTING` → "Round [N] — Vote"
 - `REVOTE` → "Round [N] — Revote"
 - `VOTE_REVEAL` → "Revealing Votes..."
 - `TIE_DETECTED` → "It's a Tie"
 - `TIEBREAKER_REVEAL` → "Tiebreaker"
 - `BANISH_REVEAL` → "Banishment"
 - `NIGHT` → "Night [N]"
 - `MORNING` → "Morning [N]"
 - `CHALLENGE` → "Challenge"
 - `CHALLENGE_RESULT` → "Challenge Result"
- Right: A compact alive player count — e.g. "8 alive" — with a small icon. Tapping/clicking this expands the player roster panel (see below).

Collapsible player roster panel: Slides down from the HUD when the alive count is tapped. Shows all players as compact cards (name + alive/dead status). Dead players are shown greyed and struck-through. Traitors can see a dagger icon next to fellow Traitors. Faithful see no role information on other players. Tapping again collapses the panel. Panel overlays the content below (does not push it down). Close automatically when phase changes.

2. Action prompt bar

Directly below the HUD, render a contextual one-liner that tells the player exactly what they need to do right now. This is a second fixed element, ~40px tall, with a subtle background.

Map per phase and role:

Phase	Role	Prompt
<code>ROUNDTABLE</code>	Any (alive)	"Discuss. Who do you suspect?"

ROUNDTABLE	Any (dead)	"You have been eliminated. Watch silently."
VOTING	Alive, not voted	"Cast your vote — who is a Traitor?"
VOTING	Alive, voted	"Vote cast. Waiting for others... ([X]/[Y] voted)"
VOTING	Dead	"You cannot vote."
REVOTE	Alive, not voted	"Tied vote — choose between the tied players."
VOTE_REVEAL	Any	"Votes are being revealed..."
BANISH_REVEAL	Any	"[Name] has been banished."
NIGHT	TRAITOR (alive)	"Choose your victim."
NIGHT	FAITHFUL (alive)	"The Traitors are choosing their victim. Sleep well."
NIGHT	Any (dead)	"You watch from the shadows."
MORNING	Any	"A new day begins."
CHALLENGE	Not answered	"Complete the challenge before time runs out."
CHALLENGE	Answered	"Answer submitted. Waiting for others."

Prompt text is derived from props — no server call needed. The HUD receives `phase`, `currentPlayerId`, `players`, `votes`, `currentRound` as props from `App.tsx`.

3. Wire HUD into App.tsx

In `client/src/App.tsx`, render `<HUD />` and the action prompt bar above the `phase` component, conditionally (not shown in `LOBBY`, `ROLE_ASSIGN`, `ROLE_REVEAL`, `GAME_END`). Pass the necessary props from the existing game state already held in `App.tsx`'s state.

Do not refactor `App.tsx`'s state management — only add the HUD rendering.

4. Phase transition flash

When the phase changes, briefly flash the action prompt bar with a highlight colour (`#c9a84c` at 40% opacity) for 600ms using a CSS animation. This gives players a clear visual signal that something changed — especially important on mobile when they may have scrolled.

Implement using a CSS `@keyframes` flash animation triggered by adding/removing a class on phase change.

Constraints

- HUD must not break existing component layouts — use `position: fixed` and add `padding-top` to the main content area to compensate
- Role information visible in the HUD must be strictly correct — Faithful players must never see other players' roles
- CSS Modules only — no inline styles, no Tailwind
- TypeScript strict mode must be satisfied
- Do not modify any server-side files

Definition of done

- HUD renders correctly in all in-game phases
 - Role badge shows correct role with correct colour
 - Player roster panel opens and closes correctly
 - Action prompt is accurate for every phase/role combination
 - Phase transition flash fires on every phase change
 - TypeScript compiles cleanly, no console errors
-

PROMPT 2: Mobile Polish — Full Audit & Fix

You are working on **Betrayal Game** (Node.js + TypeScript backend, React 19 + Vite frontend, CSS Modules). The game is primarily played on mobile phones (players hold their phone while sitting around a table or on a video call).

The problem

The game was built desktop-first and has not been audited for mobile. The likely issues are: touch targets too small, chat overlapping game content, fixed-height elements not accounting for dynamic viewport height (iOS Safari `100vh` bug), input fields being obscured by the keyboard, and text being too small to read comfortably at arm's length.

What needs to be done

Do a complete mobile audit and fix across all components. The target viewport is 375px wide (iPhone SE), and everything must work correctly on iOS Safari and Android Chrome.

1. Viewport and layout foundations

In `client/src/index.css` :

- Replace all uses of `height: 100vh` with `height: 100dvh` (dynamic viewport height — fixes iOS Safari address bar issue). Where `100dvh` is not supported, fall back to `100vh` using `@supports`.
- Ensure `<meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">` is set in `client/index.html` (add if missing — `maximum-scale=1` prevents iOS auto-zoom on input focus).
- Set `touch-action: manipulation` on all interactive elements globally to eliminate the 300ms tap delay.

2. Touch target audit

Every tappable element must meet a **minimum 44×44px touch target**. Audit every button, player card, vote target, and chat submit button across:

- `Lobby.tsx` — join/create buttons, room code input
- `Voting.tsx` — player selection cards
- `NightPhase.tsx` — target selection cards
- `ChatBox.tsx` — send button, message input
- `Challenge.tsx` — answer buttons/input
- `Timer.tsx` — no tappable elements, but verify text size

Fix any element below 44px by adding `min-height: 44px` and `min-width: 44px` (or `padding` to achieve the same) in the relevant CSS Module.

3. ChatBox — mobile layout fix

`ChatBox.tsx` is the most complex mobile problem. Fix:

- The chat input must not be obscured when the iOS keyboard appears. Use `position: sticky; bottom: 0` on the input container rather than `position: fixed`. Ensure the message list has `overflow-y: auto` and scrolls independently.

- Add `inputmode="text"` and `enterkeyhint="send"` to the chat `<input>` element so mobile keyboards show a "Send" button on the return key.
- Wire the `enterkeyhint` — pressing Enter/Send on mobile should submit the message (already works on desktop; ensure `onKeyDown` handler fires correctly on mobile).
- When the chat input is focused, scroll the message list to the bottom automatically.
- The chat panel, when open, should take no more than 40% of the screen height on mobile (375px width), leaving the game content visible above it.

4. Voting.tsx — player card layout

On mobile, player cards must:

- Display in a 2-column grid below 480px width (use CSS Grid with `repeat(2, 1fr)`)
- Each card must be at least 80px tall with the player name and vote status clearly readable at 16px minimum font size
- The selected/voted state must be visually obvious — don't rely on colour alone (add a checkmark icon or border change)
- The "You have already voted" state should show clearly which player you voted for

5. NightPhase.tsx — Traitor target selection

On mobile for Traitors:

- Target player cards must be the same 2-column grid as Voting
- The murder vote confirmation step (if any) must be a large, full-width button, not a small inline element
- For Faithful players in Night phase, the "waiting" screen must be calm and readable — not a blank screen. Show: a dark background, the round number, and the message "The Traitors are deciding. Try to look innocent." in large centred text.

6. Lobby.tsx — room code sharing

On mobile, players need to share the room code with friends. Add a "Copy Code" button next to the room code display that:

- Calls `navigator.clipboard.writeText(sessionId)`
- Shows a brief "Copied!" confirmation (inline text swap for 2 seconds, no toast library)

- Falls back gracefully if clipboard API is unavailable (show the code in a selectable `<input readonly>`)

7. Typography and spacing

In `client/src/index.css` :

- Base font size: `16px` minimum (never smaller — prevents iOS auto-zoom)
- Line height: `1.5` for body text, `1.2` for headings
- All game-state text (phase names, player names, role reveals) must be at least `18px`
- Increase spacing between list items and cards — minimum `8px` gap

Constraints

- Do not introduce any new npm packages — use only existing dependencies
- CSS Modules only — fix styles in the relevant `.module.css` files
- Do not restructure component logic — this is styling and layout only (except the clipboard copy button and keyboard submit, which require minor JS)
- Test every fix against the 375px viewport mentally — document in code comments where a fix addresses a specific mobile issue
- TypeScript strict mode must remain satisfied

Definition of done

- No element has a touch target below 44×44px
- Chat input is not obscured by iOS keyboard
- Voting grid renders 2 columns at 375px width
- Room code has a working copy button
- `100dvh` used throughout with `100vh` fallback
- Night phase shows a meaningful waiting screen for Faithful players
- TypeScript compiles cleanly

PROMPT 3: Host Control Panel

You are working on **Betrayal Game** (Node.js + TypeScript backend, React 19 + Vite frontend, CSS Modules). The host player has special privileges — they can force-resolve votes (`C2S_FORCE_RESOLVE_VOTING`) and control phase progression. Currently, host controls are mixed into the regular game UI with no dedicated view.

The problem

Running a game as host requires constant attention across multiple components. There is no single place to see: who hasn't voted yet, how long is left on the timer, which phase buttons to press next, and whether the game is in a healthy state. In a real playtest with 8–12 people, this friction causes the host to fumble and slow the game down.

What needs to be done

1. Host Control Panel component

Create `client/src/components/HostPanel.tsx` and `HostPanel.module.css` .

The panel is visible **only to the host player** and renders as a slide-in drawer from the bottom of the screen, toggled by a floating “ Host” button (bottom-right, above the chat). It overlays content — it does not push the layout.

The panel has the following sections, rendered based on the current phase:

Always visible — Game Status:

- Current phase (human-readable)
- Round number
- Player count: alive / total
- Timer status: time remaining if a timer is active, “No timer” if not

Phase-specific controls:

LOBBY :

- “Start Game” button (sends `C2S_START_GAME`) — disabled if fewer than `minPlayers` connected, with explanation: “Need [N] more players”
- List of connected players with connection status indicator (green dot = connected)

ROUNDTABLE :

- “End Discussion & Start Vote” button (sends `C2S_START_VOTING`)

- List of connected players — highlight any who appear disconnected (`isConnected: false`)

VOTING / REVOTE :

- Vote progress: "[X] of [Y] players have voted"
- List of who has NOT voted yet (names only — do not show who they voted for)
- "Force Resolve Votes" button (sends `C2S_FORCE_RESOLVE_VOTING`) — shown only when at least one vote has been cast. Include a warning: "Unvoted players will receive auto-votes."
- Timer countdown if active

VOTE_REVEAL :

- "No action needed — vote reveal is automatic" message
- Option to skip to result: "Skip Reveal" button (sends `C2S_BANISH_PLAYER`) — with confirmation: "Are you sure? This skips the reveal animation."

BANISH_REVEAL / TIE_DETECTED / TIEBREAKER_REVEAL :

- "Continue to Night" button (sends `C2S_START_NIGHT`)
- Banished player name and role displayed

NIGHT :

- Murder vote progress: "[X] of [Y] Traitors have voted" (count only — host does not see who was targeted)
- "Force Resolve Night" button (sends `C2S_RESOLVE_MURDER`) — with confirmation dialog
- Timer countdown if active

MORNING :

- Murdered player name (if any) — "No murder tonight" if none
- "Continue to Day" button (sends `C2S_CONTINUE_TO_DAY`)

CHALLENGE / CHALLENGE_RESULT :

- Challenge type displayed
- Answer progress: "[X] of [Y] players have answered"
- "Continue to Roundtable" button (sends `C2S_CONTINUE_TO_ROUNDTABLE`) — available

after at least one answer

Always visible — Danger Zone (collapsed by default, expand on tap):

- “Transfer Host” — dropdown of connected players, confirm button. Sends a new `C2S_TRANSFER_HOST` event (add this event to the types if it doesn’t exist — server should call `game.transferHost` and broadcast `S2C_HOST_TRANSFERRED` with the new host’s player ID).
- “End Game Early” — sends `C2S_END_GAME_EARLY` (add this event). Server sets `winner: null`, phase to `GAME_END`, and broadcasts `S2C_GAME_END` with a `reason: 'HOST_ENDED'` field.

2. New server-side events

Add to `src/game/types.ts` and handle in `src/ws/router.ts`:

```
// Client → Server
{ type: 'C2S_TRANSFER_HOST', payload: { newHostId: string } }
{ type: 'C2S_END_GAME_EARLY', payload: Record<string, never> }

// Server → Client
{ type: 'S2C_HOST_TRANSFERRED', payload: { newHostId: string; newHostName: string
```

`C2S_TRANSFER_HOST`: validate that sender is current host, call `game.transferHost(gameState, newHostId)`, save, broadcast `S2C_HOST_TRANSFERRED`.

`C2S_END_GAME_EARLY`: validate sender is host, set `phase: 'GAME_END'`, `winner: null`, broadcast `S2C_GAME_END` with `reason: 'HOST_ENDED'` and `history: gameState.history`.

3. Host panel state

The host panel reads from the same game state already present in `App.tsx`. It does not need its own WebSocket subscription. Pass props down: `gameState`, `currentPlayerId`, `isHost`, `onSendEvent` (a callback to send a WS message).

When the host panel sends an event via `onSendEvent`, the `useWebSocket` hook handles it — no direct WebSocket access in `HostPanel.tsx`.

4. Confirmation dialogs

Any destructive action (Force Resolve, Skip Reveal, End Game Early) must show an inline confirmation within the panel — not a browser `confirm()` dialog. Pattern: button press shows “Are you sure? [Confirm] [Cancel]” inline. Confirm sends the event. Cancel resets.

Use local React state for this — no modal library.

5. Styling

The "⚙️ Host" toggle button:

- `position: fixed, bottom: 80px, right: 16px` (above chat if chat is open)
- Background: `#c9a84c` (gold), text black, 44px minimum height, `border-radius: 22px`
- Shows a red dot indicator when a host action is needed (e.g. voting is complete, night votes are complete)

The panel itself:

- Slides up from the bottom: `transform: translateY(100%) → translateY(0)`, 300ms ease-out
- Max height: 70% of screen, scrollable internally
- Background: `#1a1a1a`, border-top: 2px solid `#c9a84c`
- Section headings in gold, body text in `#f0e6d3`
- Destructive buttons (Force Resolve, End Game) in `#8b1a1a` red

Constraints

- Host panel is completely invisible to non-host players — render conditionally on `isHost` in `App.tsx`, never send to server
- All phase progression buttons in the host panel must replace equivalent buttons that may exist in other components — audit and remove duplicates from `Voting.tsx`, `NightPhase.tsx`, `Lobby.tsx` so controls live in one place
- Do not break the existing non-host player experience
- TypeScript strict mode must be satisfied

Definition of done

- Host panel opens and closes via the floating button
- Every phase shows the correct controls and status information
- Force Resolve, Transfer Host, and End Game Early all work end-to-end
- Confirmation dialogs work for all destructive actions

- Red dot indicator appears when host action is needed
 - Non-host players see no host UI elements
 - TypeScript compiles cleanly
-

PROMPT 4: Onboarding & Role Tutorial

You are working on **Betrayal Game** (Node.js + TypeScript backend, React 19 + Vite frontend, CSS Modules). First-time players currently receive no explanation of the rules before the game starts.

The problem

The game has a non-obvious ruleset. New players don't know: what Traitors vs Faithful means, how voting works, what happens at night, or what the win conditions are. In a friend group of 10–20, there will almost always be first-timers. Currently the host has to verbally explain the rules before every game, which takes 3–5 minutes and kills momentum.

What needs to be done

1. Rules modal in the Lobby

Add a "How to Play" button to `Lobby.tsx` (below the join/create controls). Tapping it opens a full-screen modal (CSS only, no library).

The modal has five swipeable/tappable pages — navigate with Next/Back buttons and a dot progress indicator:

Page 1 — The Setup Title: "Two sides. One truth." Body: "You'll be secretly assigned as either a Faithful or a Traitor. Faithful players don't know who the Traitors are. Traitors know each other." Visual: Two silhouettes — one lit, one shadowed.

Page 2 — The Roundtable Title: "Discuss. Suspect. Accuse." Body: "Each round begins with open discussion. Talk, probe, and argue about who might be a Traitor. At the end of discussion, everyone votes to banish one player." Visual: A simple table icon with speech bubbles.

Page 3 — The Vote Title: "The group decides." Body: "All alive players vote to banish one person. The player with the most votes is banished — and their role is revealed. If it's a Faithful, the Traitors win ground. If it's a Traitor, the Faithful are safer." Visual: A ballot icon with a checkmark.

Page 4 — The Night Title: "While you sleep..." Body: "After each vote, night falls. Traitors secretly coordinate to eliminate one Faithful player. The Faithful can only wait. In the morning, someone may be gone." Visual: A moon/star icon.

Page 5 — Winning Title: "How it ends." Body: "Faithful win by banishing all Traitors. Traitors win when they equal or outnumber the Faithful — they've taken control." Visual: Two outcome icons — shield (Faithful) and dagger (Traitor).

All visuals are CSS-only illustrations — no image files. Simple geometric shapes using `border-radius`, `clip-path`, or `::before` / `::after` pseudo-elements.

The modal can be dismissed at any time with an X button or by tapping the backdrop.

Remember progress in `localStorage` key `betrayal_rules_seen` — if `true`, show "How to Play" as a smaller secondary link rather than a prominent button.

2. Role reveal enhancement

The existing `RoleReveal.tsx` component shows the player their role. Extend it to include a role-specific briefing — shown for 8 seconds before the player can dismiss it (prevents accidental skipping).

For FAITHFUL players:

- Large "FAITHFUL" title in slate blue
- Below: "You don't know who the Traitors are. Observe carefully. Trust your instincts. Vote wisely."
- Then: "Your goal: banish all Traitors before they outnumber you."
- A 8-second countdown bar before the "I understand" button appears

For TRAITOR players:

- Large "TRAITOR" title in crimson
- Below: "You know your fellow Traitors. Blend in. Deflect suspicion. At night, choose your victim together."
- If there are other Traitors: "Your fellow Traitors: [Name], [Name]"
- Then: "Your goal: equal or outnumber the Faithful."
- A 8-second countdown bar before the "I'm ready" button appears

The countdown bar is a CSS animation — a bar that drains from full width to 0 over 8 seconds using `@keyframes`. After 8 seconds, the button fades in (`opacity: 0 → 1`),

500ms).

3. Contextual phase intro cards

When a phase starts for the first time in a game session, briefly show a dismissible card (3-second auto-dismiss, or tap to dismiss) explaining what's about to happen. "First time" means: track seen phases in a `useRef` set — show the card only on the first occurrence of each phase per session.

Phase	Card title	Card body
VOTING (first time)	"Time to vote"	"Choose who you think is a Traitor. All votes are revealed at once."
NIGHT (first time)	"Night falls"	"Traitors: coordinate your choice. Faithful: stay calm."
REVOTE (first time)	"Tied vote"	"The vote was tied. Vote again between the tied players only."
CHALLENGE (first time)	"Challenge!"	"A challenge has been issued. The winner earns a shield — protection from one banishment or murder."
MORNING (first time)	"Morning arrives"	"Find out if someone was murdered in the night."

Cards render as a centred overlay with a semi-transparent backdrop. They auto-dismiss after 3 seconds or on tap. Use CSS `@keyframes` — slide up, pause, slide down.

Implement in a `PhaseIntroCard.tsx` component that `App.tsx` renders conditionally.

Constraints

- How to Play modal: all visuals must be CSS-only — no SVGs from files, no image tags
- Role reveal countdown: must be a CSS animation — not a `setInterval` counter
- Phase intro cards: must not block game interaction if the player dismisses early
- `localStorage` key for rules seen must use the `betrayal_` prefix
- TypeScript strict mode must be satisfied
- Do not modify any server-side files

Definition of done

- How to Play modal renders all 5 pages with navigation
 - Modal is not shown prominently to returning players (rules_seen flag works)
 - Role reveal shows role-specific briefing with 8-second countdown bar
 - Phase intro cards appear on first occurrence of each phase
 - Cards auto-dismiss after 3 seconds
 - TypeScript compiles cleanly, no console errors
-

PROMPT 5: Sound & Haptic Overhaul

You are working on **Betrayal Game** (Node.js + TypeScript backend, React 19 + Vite frontend, CSS Modules). `Howler.js` is already installed and wired up via

`client/src/contexts/SoundContext.tsx` and `client/src/hooks/useSounds.ts`. Mobile haptic feedback exists in `client/src/utils/haptics.ts`.

The problem

Sound and haptics are partially implemented but not systematically applied. The result is an inconsistent audio experience — some actions have sounds, others don't, and the sounds don't reinforce the drama of the game's key moments. World-class social deduction games use audio as a primary storytelling tool: the sound of a vote being cast, the silence of the night phase, the sting of a banishment.

Audit first

Before writing any code, read the existing `SoundContext.tsx`, `useSounds.ts`, and `haptics.ts` to understand exactly what sounds are currently defined, which events trigger them, and what audio files exist in the project. List every existing sound and where it is triggered. Then implement the following as an extension — do not remove existing sounds unless they are clearly broken.

Sound design spec

All new sounds must be **generated programmatically using the Web Audio API** — do not require any new audio files. Create a utility `client/src/utils/audioSynth.ts` that exports synthesised sound functions. These are used alongside Howler (Howler handles file-based sounds; the synth handles procedurally generated ones).

```
// All functions return void, play immediately, use the global AudioContext
```

```

export function playDrum(intensity: 'soft' | 'hard'): void
// A single drum hit — soft: low sine wave, 80ms decay; hard: noise burst, 40ms d

export function playChime(note: 'low' | 'mid' | 'high'): void
// A simple sine wave chime — low: 220Hz, mid: 440Hz, high: 880Hz, 600ms decay

export function playRiser(durationMs: number): void
// A tension riser — white noise with a high-pass filter sweeping up over duratio

export function playStab(): void
// A sharp orchestral stab — two sine waves (220Hz + 440Hz) at high volume, 200ms

export function playHeartbeat(bpm: number): void
// Two drum hits (lub-dub pattern) at the given BPM, plays once

export function stopAllSynth(): void
// Stops any currently playing synth sounds

```

Implement `getAudioContext()` as a lazy singleton — create the `AudioContext` on first call (after a user gesture, to comply with browser autoplay policy). If `AudioContext` is not available, all functions are no-ops.

Sound event mapping

Apply sounds to the following game events. Wire them in the relevant React components by calling `useSounds()` or the synth functions directly. Each entry specifies the trigger, the sound, and the haptic pattern.

Phase transitions:

Event	Sound	Haptic
Game starts (<code>S2C_GAME_STARTED</code>)	<code>playRiser(3000)</code>	heavy
Role revealed (<code>S2C_ROLE_REVEAL</code>) — TRAITOR	<code>playStab()</code> then silence	heavy + 500ms pause + heavy
Role revealed (<code>S2C_ROLE_REVEAL</code>) — FAITHFUL	<code>playChime('high')</code>	light
Voting starts (<code>S2C_VOTING_STARTED</code>)	<code>playDrum('hard')</code>	medium
Night starts (<code>S2C_NIGHT_STARTED</code>)	<code>playRiser(2000)</code> at low volume	light

Morning starts (<code>S2C_MORNING_STARTED</code>) — murder occurred	<code>playStab()</code>	heavy
Morning starts (<code>S2C_MORNING_STARTED</code>) — no murder	<code>playChime('mid')</code>	light

Vote reveal sequence:

Event	Sound	Haptic
Each vote step (<code>S2C_VOTE_REVEAL_STEP</code>)	<code>playDrum('soft')</code>	none
Final vote (<code>S2C_VOTE_REVEAL_COMPLETE</code>)	<code>playDrum('hard')</code> then <code>playRiser(1000)</code>	medium
Player banished (<code>S2C_PLAYER_BANISHED</code>)	<code>playStab()</code>	heavy
Tie detected (<code>S2C_TIE_DETECTED</code>)	Three <code>playDrum('soft')</code> at 200ms intervals	medium

Player actions:

Event	Sound	Haptic
Own vote cast (<code>C2S_SUBMIT_VOTE sent</code>)	<code>playChime('low')</code>	light
Another player voted (<code>S2C_VOTE_SUBMITTED</code>)	<code>playDrum('soft')</code>	none
Murder vote cast by Traitor	<code>playDrum('hard')</code>	medium
All murder votes in (<code>S2C_MURDER_RESOLVED</code>)	<code>playRiser(1000)</code>	heavy

Timer urgency:

When a timer is active and has ≤ 10 seconds remaining, play `playHeartbeat(120)` on each second tick. Stop when the timer expires or the phase changes. This is the most important audio cue in the game — it creates real pressure.

Implement the timer heartbeat in `Timer.tsx`. The `Timer` component already tracks remaining time — add a `useEffect` that calls `playHeartbeat(120)` when `remainingSeconds <= 10` changes to true, and plays once per second using a `setInterval` that is cleaned up on phase change.

Mute control:

Add a mute toggle button to the HUD (from Prompt 1) — a small speaker icon () () , top-right of the HUD bar. Muted state stored in `localStorage` key `betrayal_muted` . When muted: all synth functions are no-ops, and set Howler's global volume to 0. When unmuted: restore Howler volume to 1.

Read mute state from `localStorage` on app load. If a player's first interaction with the game triggers the `AudioContext`, ensure sounds don't fire before that first gesture.

Haptic patterns reference

The existing `haptics.ts` utility likely exports a `vibrate(pattern)` function. Use the following patterns:

- `light` : [10]
- `medium` : [30]
- `heavy` : [50]
- `heavy + pause + heavy` : [50, 500, 50]

If the existing haptics utility uses different naming, adapt accordingly — do not change the utility's API.

Constraints

- All new sounds must be Web Audio API synthesised — no new audio files
- Sounds must not play before a user gesture (`AudioContext` autoplay policy)
- Mute state must persist in `localStorage` across sessions
- Do not remove or break existing Howler sounds
- The heartbeat must stop cleanly when a phase ends — no orphaned intervals
- TypeScript strict mode must be satisfied
- All synth functions must be no-ops if Web Audio API is unavailable (graceful degradation)

Definition of done

- All 15+ sound events are wired and audible in a real game session
- Timer heartbeat plays in the last 10 seconds of any timed phase

- Mute toggle works and persists across page reloads
- Role reveal sounds are clearly distinct between Traitor and Faithful
- Vote reveal has a rhythmic drum-per-vote feel
- Banishment has a definitive audio sting
- No sounds play before first user interaction
- TypeScript compiles cleanly, no console errors